(a) Why is Gradient Descent a reasonable method for finding local minima to a function? In particular, which is the basic idea of this algorithm, how is it moving towards a local mimimum?

Gradient Descent is reasonable because it uses information about the slope of a function to guide the search toward a minimum. The fundamental idea is:

- The gradient (or derivative, in one dimension) tells us the direction of steepest increase. Therefore, moving in the opposite direction of the gradient takes us toward lower function values.
- Each update nudges the current point slightly downhill. Over many iterations, these small steps gradually lead the algorithm to a local minimum, as long as the step size is appropriate and the function is reasonably well-behaved. This makes Gradient Descent effective, especially when the function is too complicated to optimize analytically.

(b) Why is auto-differentiation very important/useful in the context of Gradient Descent when the function to be minimized is very complex to evaluate and differentiate?

Auto-differentiation is useful because it automatically computes derivatives, even for functions that are complicated, long, or composed of many nested operations.

Doing the derivatives manually is error-prone and impossible sometimes. Auto-differentiation ensures:

- exact gradients
- fast computation
- correct handling of the computation graph

Without auto-differentiation, running Gradient Descent on large or complex models would be extremely difficult.

(c) What is the name of the second relatively new computational framework, similar to PyTorch, which was also designed with optimization of ANN parameters in mind? This alternative framework was developed by Google in is currently also very popular.

The second popular framework, developed by Google, is TensorFlow.

(d) Do you think you understand the key idea presented in Appendix C below regarding the key idea of auto-differentiation? What is the basic principle the auto-differentiation methodology is relying on?

Yes, the main idea of auto-differentiation is that any complicated function can be broken down into a sequence of simple, elementary operations. Each of these small operations has a known derivative rule.
Auto-differentiation works by:

- Recording the sequence of operations in a computational graph during the forward pass.
- Applying the chain rule backward through the graph to compute the derivative/gradient of the final output with respect to each variable.

(e) What did you personally find interesting and/or useful and/or informative with the assignment, and why? If you actually did not find it enlightening, in that cas,e we would like to know about why.

I found it interesting to see how PyTorch handles auto-differentiation behind the scenes. Before this assignment, Gradient Descent felt abstract, but implementing it manually helped me understand:

- How gradients flow through a computational graph
- Why PyTorch requires_grad=True
- How the .backward() call builds and uses accumulated gradients
- Why do we need torch.no_grad() during updates

Plotting the trajectories also gave me a better intuition for how different step sizes affect convergence. Overall, the assignment connected the theory of optimization with practical numerical implementation, which was very helpful.

(f) What did you find frustrating and/or difficult, and/or unnecessary, and why?

What I found most frustrating was understanding some of the technical PyTorch requirements, such as:

- Why x = x - eta*x.grad does not work
- Why gradients accumulate by default
- why updates must be inside a torch.no_grad() block

However, after reading the explanations and experimenting, the logic became clearer.

(g) What would you suggest as an improvement of this assignment for next years students, and why?

One improvement would be to include a short, step-by-step tutorial on the PyTorch gradient mechanics. This would give students a clearer starting point and reduce time spent trying to understand the technicalities rather than focusing on the main learning goals.

(h) Did you spend more than 20 hours to complete this exercise? Why?

Yes, I spent more than 20 hours. Most of the time went into:

- Understanding the PyTorch gradient machinery
- Debugging small issues, such as wrong tensor updates
- Experimenting with step sizes and trajectories
- Making sure I correctly implemented the plotting and conversion between tensors and NumPy

The conceptual part was clear, but implementing everything correctly in PyTorch required extra time and careful debugging.

Thank you!